Delivery semantic (MOM)	Descrizione	Meccanismi in uso	Supporta Non Idempotent
At-least once	non si perdono messaggi, ma potrebbero essere consegnati più volte	Consumer: ack e duplication filtering Producer: timer sul ack per riconsegna	Si, dato che il consumer implementa duplication filtering
At-most once	si potrebbero perdere i messaggi e non si riconsegnerebbero mai	Nessuno	No
Exactly once	messaggio consegnato una volta sola	il MOM implementa duplication filtering su i messaggi del producer	Si, grazie a duplication filtering
Transaction based	I messaggi sono eliminati solo dopo che sono stati letti e processati dal consumer	MOM e consumer si comportano come in presenza di una transazione e rispettano ACID	Si, grazie al ACID
Timeout based	Si assicura che i messaggi siano consegnati almeno una volta prevenendo il doppio consumo	un messaggio appena letto diventa invisibile e se non cancellato torna visibile dopo che finisce il visibility timeout	Si, dato che se il consumer fallisce il processamento, torna a leggere il messaggio originale

Delivery semantic (RPC)	Descrizione	Meccanismi in uso	Supporta Non Idempotent
At-least once	servizio eseguito almeno una volta	Client: request retry Server: nessuno	No, dato che c'è request retry e nessun duplication filtering
At-most once	il servizio è eseguito at-most once	Client: request retry Server: duplication filtering e response retransmission	Si, grazie a duplication filtering e response retransmit
Exactly once	il servizio è eseguito exactly once	Quelli base non bastano, serve anche fault tolerance con write-ahead logging	Si, grazie a duplication filtering, response retransmit e WAL

Sistema MOM	Delivery semantic	Delivery model	Queue type	Replication	Pattern di riferimento	Message retention
Amazon SQS (messaggi)	Timeout based, con visibility timer	Pull: short poll, chiedo a un subset di server. Long poll chiedo a tutti i server	Standard: ordine non garantito e duplicati. FIFO: ordine e nessun duplicato garantiti	Garantita. Nella stessa regione, i messaggi sono su più server	Message queue	I consumer cancellano i messaggi dopo max 14 giorni
Kafka (stream di eventi)	At-most: se acks=0. At-least: se acks=1. Exactly: se acks=all.	Pull	FIFO, con ordine garantito all'interno di una partizione	Si, con replication factor	Publisher-subscriber	I messaggi restano nella coda in base alla dimensione o al retention time
RabbitMQ	At-most (default): autoAck=True. At-least: autoAck=False	il MOM implementa duplication filtering su i messaggi del producer	FIFO con ordine garantito a queue level	Si, viene supportata attraverso i cluster e le quorum queues	Entrambi i pattern sono realizzabili	Dopo la lettura i messaggi vengono cancellati dalle code

Tecnologia	Access	Location	Replication	Failure	Concurrency	Sincronia
Sun RPC	Parziale, serve sapere il nome della procedura e i parametri	Parziale, tramite il port mapper serve solo hostname	No. Ma se introduco un proxy server side posso distribuire il carico	Non è garantita	No, le procedure eseguono in mutua esclusione	No, supporta solo procedure sincrone
Java RMI	Si, attraverso pattern proxy, permette di manipolare oggetti remoti	Parziale, tramite RMI registry che mantiene gli oggetti remoti	No. Ma se introduco un proxy server side posso distribuire il carico	Non è garantita	Parziale, gli oggetti devono essere invocati in contesto thread-safe	No, supporta solo procedure sincrone
RPyC	Si, con pattern proxy via netrefs, permette di manipolare oggetti remoti	Non è garantita, serve hostname e porta	No. Ma se introduco un proxy server side posso distribuire il carico	Non è garantita	Parziale, ci sono diverse configurazioni ma vanno gestiti i lock	Si, supporta procedure asincrone
Go RPC (net/rpc)	Parziale, serve sapere il nome della procedura e i parametri	Non è garantita, serve hostname e porta anche se si usa il registrer nel server	No. Ma se introduco un proxy server side posso distribuire il carico	Non è garantita	Parziale, si usano le goroutine ma vanno gestite	Si, supporta procedure asincrone
gRPC	Garantita, tramite Protobuf	Non è garantita, serve hostname e porta	Parziale, tramite un Load balancer esterno	Parziale, permette di fare retry e request hedging se configurato	Parziale, la gestione è application dependent	Si, supporta procedure asincrone